

Week-5 (DAY-22) Hands-On Keerthivasan R

Problem 1 – Level 1

Scenario:

A small organization wants to store simple log messages into a text file using a C# console application.

Requirements:

1. Accept a message from the user.
2. Write the message into a file using FileStream.
3. Append multiple messages to the same file.
4. Display confirmation after writing the data.

Technical Constraints:

- Use FileStream class.
- Use appropriate FileMode and FileAccess.
- Implement exception handling for file access errors.

Expectations:

The application should successfully write user messages to the file and allow multiple entries.

Learning Outcome:

Students will learn how to create and write data into files using FileStream.

CODE:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace HandsOnDAY22
{
    internal class FileStream_P01
    {
        static void Main(string[] args) {

            try
            {
                Console.Write("Enter message: ");
                string message = Console.ReadLine();

                using (FileStream fs = new
FileStream(@"C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-22\log.txt",
FileMode.Append,
```

```

        FileAccess.Write))
    {
        byte[] data = Encoding.UTF8.GetBytes(message + Environment.NewLine);
        fs.Write(data, 0, data.Length);
    }

    Console.WriteLine("Message written successfully!");
}
catch (Exception ex)
{
    Console.WriteLine("Error: " + ex.Message);
}
}
}
}
}

```

OUTPUT:

```

Enter message: First Message
Message written successfully!

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-22
\HandsOnDAY22\HandsOnDAY22\bin\Debug\net10.0\Ha
ndsOnDAY22.exe (process 1840) exited with code
0 (0x0).
Press any key to close this window . . .|

```

Problem 2 – Level 1:

Scenario:

An administrator wants to check file properties stored in a particular folder for auditing purposes.

Requirements:

1. Accept a folder path from the user.
2. Display file name, file size, and creation date.

3. Count and display the total number of files.

Technical Constraints:

- Use FileInfo class.
- Handle invalid directory paths.

Expectations:

The program should list file details clearly in the console.

Learning Outcome:

Students will understand how to retrieve file metadata using FileInfo.

CODE:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace HandsOnDAY22
{
    internal class FileInfo_P02
    {
        static void Main(string[] args) {

            try
            {

                Console.Write("Enter folder path: ");
                string path = Console.ReadLine();
                Console.WriteLine();

                DirectoryInfo dir = new DirectoryInfo(path);

                if (!dir.Exists)
                {
                    Console.WriteLine("Directory not found");
                    return;
                }

                FileInfo[] files = dir.GetFiles();
                int count = 0;

                foreach (FileInfo file in files)
                {
                    Console.WriteLine($"Name: {file.Name}");
                    Console.WriteLine($"Size: {file.Length}bytes");
                    Console.WriteLine($"Created: {file.CreationTime}");
                    Console.WriteLine("-----");
                    count++;
                }
                Console.WriteLine($"Total Files: {count}");
            }
            catch (Exception e) {
                Console.WriteLine("Error: " + e.Message);
            }
        }
    }
}
```

}

OUTPUT:

```
Enter folder path: C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-22

Name: log.txt
Size: 46bytes
Created: 17-03-2026 16:26:36
-----
Name: Week-5 (DAY-2) - Python Track.docx
Size: 43304bytes
Created: 17-03-2026 16:10:47
-----
Name: ~$ek-5 (DAY-2) - Python Track.docx
Size: 162bytes
Created: 17-03-2026 22:47:24
-----
Total Files: 3

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-22\HandsOnDAY22\HandsOnDAY22\bin\
Debug\net10.0\HandsOnDAY22.exe (process 6852) exited with code 0 (0x0).
Press any key to close this window . . .|
```

Problem 3 – Level 1:

Employee Performance Evaluator using Tuples and Pattern Matching (Level-1)

Scenario

A small company wants a simple program that evaluates an employee's performance based on **two inputs: monthly sales amount and customer feedback rating**.

The system should return both values together using a Tuple, and then determine the **performance category using pattern matching**.

This will help management quickly identify whether an employee is a **High Performer, Average Performer, or Needs Improvement**.

Requirements

1. Accept the following inputs from the user:
 - Employee Name
 - Monthly Sales Amount
 - Customer Feedback Rating (1–5)

2. Create a method that returns Sales Amount and Rating together using a Tuple.
3. Use pattern matching with switch expression or switch statement to categorize performance:
 - High Performer → Sales $\geq$ 100000 AND Rating $\geq$ 4
 - Average Performer → Sales $\geq$ 50000 AND Rating $\geq$ 3
 - Needs Improvement → All other cases
4. Display:
 - Employee Name
 - Sales Amount
 - Rating
 - Performance Category

Technical Constraints

- The method must return multiple values using Tuple.
- Pattern matching must be used for performance classification.
- Avoid using multiple nested if-else statements.
- Input values must be numeric and valid.

Expectations

- Students should demonstrate how to create and return tuples from a method.
- Students should use pattern matching to simplify conditional logic.
- Output should be clearly formatted.

Example Output:

Employee Name: Rahul
Sales Amount: 120000
Rating: 4
Performance: High Performer

Learning Outcome

After completing this problem, students will be able to:

- Understand how tuples return multiple values from methods.

- Apply pattern matching to simplify complex conditions.
- Write cleaner and more readable decision logic in C#.

CODE:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace HandsOnDAY22
{
    internal class EmployeePerformanceEvaluator_P03
    {
        static (double sales, int rating) GetPerformanceData(double sales, int rating)
        {
            return (sales, rating);
        }
        static void Main(string[] args) {

            try
            {
                Console.Write("Enter Employee Name: ");
                string name = Console.ReadLine();

                Console.Write("Enter Sales Amount: ");
                double sales = double.Parse(Console.ReadLine());

                Console.Write("Enter Rating (1-5): ");
                int rating = int.Parse(Console.ReadLine());

                var result = GetPerformanceData(sales, rating);

                //Pattern matching switch expression
                // This checks both values in the tuple simultaneously
                string performanceCategory = result switch
                {
                    ( >= 100000, >= 4) => "High Performer",
                    ( >= 50000, >= 3) => "Average Performer",
                    _ => "Needs Improvement" // The discard symbol '_' handles all other cases
                };

                Console.WriteLine("\n===== RESULT =====");
                Console.WriteLine($"Employee Name: {name}");
                Console.WriteLine($"Sales Amount: {result.sales}");
                Console.WriteLine($"Rating: {result.rating}");

                Console.WriteLine($"Performance: {performanceCategory}");
            }
            catch (FormatException)
            {
                Console.WriteLine("Error: Please enter valid numeric values for Sales and
Rating.");
            }
            catch (Exception ex)
            {
                Console.WriteLine($"An unexpected error occurred: {ex.Message}");
            }
        }
    }
}
```

OUTPUT:

```
Enter Employee Name: Arjun
Enter Sales Amount: 1000000
Enter Rating (1-5): 5

===== RESULT =====
Employee Name: Arjun
Sales Amount: 1000000
Rating: 5
Performance: High Performer

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-22\HandsOnDAY22\HandsOnDAY22\bin\
Debug\net10.0\HandsOnDAY22.exe (process 23756) exited with code 0 (0x0).
Press any key to close this window . . .|
```

Problem 4 – Level 2

Scenario:

A development team wants to analyze all folders inside a project directory to understand the project structure.

Requirements:

1. Accept a root directory path.
2. Display all subdirectories inside the root folder.
3. Show the number of files present in each directory.

Technical Constraints:

- Use DirectoryInfo class.
- Use loops to iterate through directories.
- Implement exception handling.

Expectations:

The application should display folder names and file counts for each directory.

Learning Outcome:

Students will learn how to navigate directories and retrieve folder information using DirectoryInfo.

CODE:

```
using System;
using System.Collections.Generic;
using System.Reflection.Metadata;
using System.Text;

namespace HandsOnDAY22
{
    internal class DirectoryInfo_P_4
    {
        static void Main(string[] args) {

            try {
                Console.Write("Enter root directory path: ");
                string path = Console.ReadLine();

                DirectoryInfo dir = new DirectoryInfo(path);

                if (!dir.Exists) {
                    Console.WriteLine("Directory not found!");
                    return;
                }

                DirectoryInfo[] subDirs = dir.GetDirectories();

                foreach (DirectoryInfo subDir in subDirs)
                {
                    int fileCount = subDir.GetFiles().Length;

                    Console.WriteLine($"Folder: {subDir.Name}");
                    Console.WriteLine($"Files Count: {fileCount}");
                    Console.WriteLine("-----");
                }
            }
            catch (Exception e) {
                Console.WriteLine("Error: " + e.Message);
            }
        }
    }
}
```

OUTPUT:

```
Enter root directory path: C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-22  
Folder: HandsOnDAY22  
Files Count: 1  
-----
```

```
C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-22\HandsOnDAY22\HandsOnDAY22\bin\  
Debug\net10.0\HandsOnDAY22.exe (process 28668) exited with code 0 (0x0).  
Press any key to close this window . . .|
```